Perceptron Learning and AND Classification

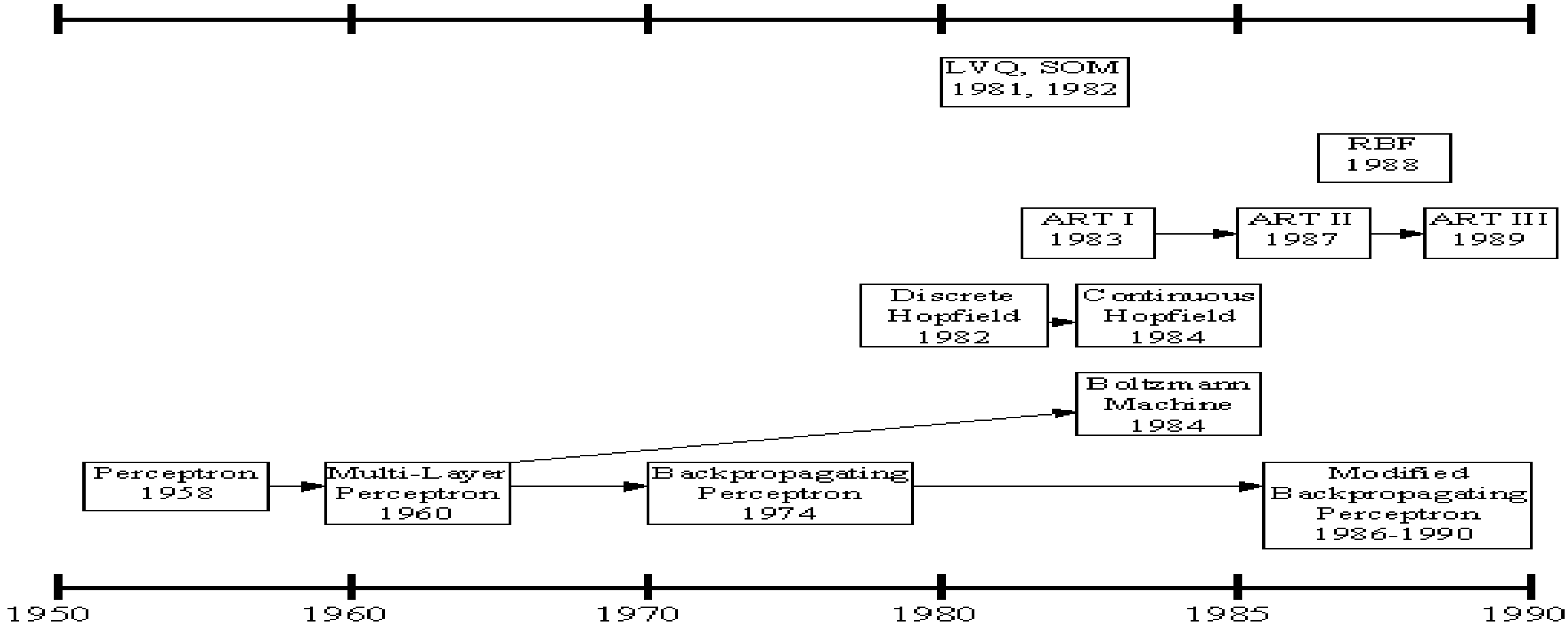


Neural Networks

- Neural Networks inspired from human central nervous system. They are based on making mistakes and learning lessons from past errors. They produces results very fast like human reflexes in contrast to learning which lasts continuous.
- history of the ANNs stems from the 1940s, the decade of the first electronic computer.
- However, the first important step took place in 1957 when Rosenblatt introduced the first concrete neural model, the perceptron. Rosenblatt also took part in constructing the first successful neurocomputer, the Mark I Perceptron. After this, the development of ANNs has proceeded as described in *Figure*.

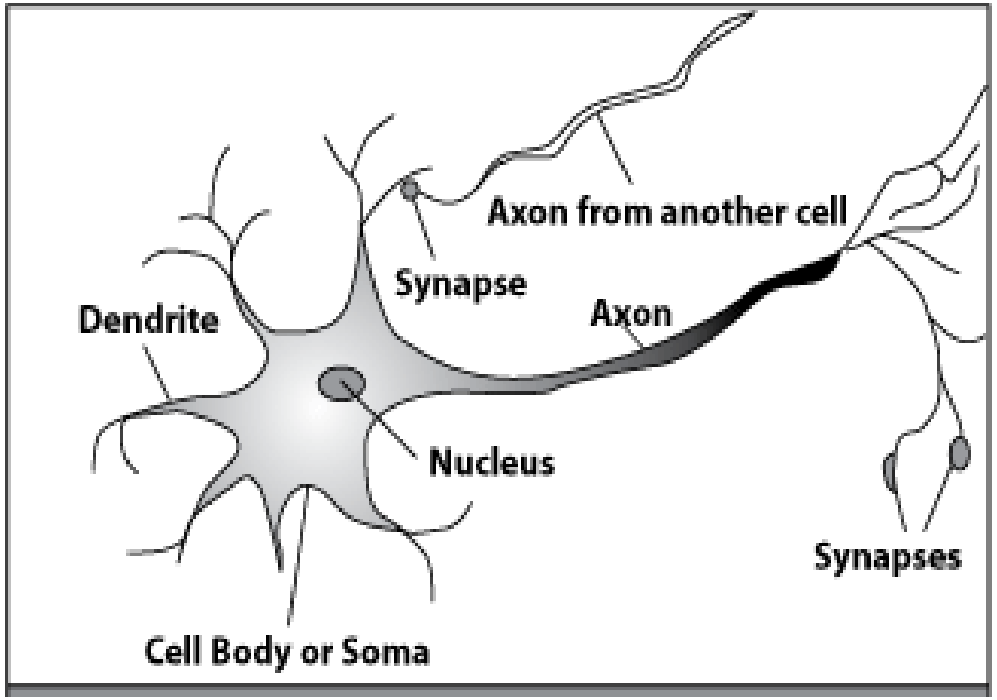
Neural Networks

- Rosenblatt's original perceptron model contained only one layer. From this, a multi-layered model was derived in 1960. At first, the use of the multi-layer perceptron (MLP) was complicated by the lack of a appropriate learning algorithm.
- In 1974, Werbos came to introduce a so-called backpropagation algorithm for the three-layered perceptron network.


LVQ, SOM
1981, 1982
RBF
1988
ART I
1983
ART II
1987
ART III
1989
Discrete
Hopfield
1982
Continuous
Hopfield
1984
Boltzmann
Machine
1984
Perceptron
1958
Multi-Layer
Perceptron
1960
Backpropagating
Perceptron
1974
Modified
Backpropagating
Perceptron
1986-1990
1950
1960
1970
1980
1985
1990

How do our brains work?

- A processing element
- Dendrites: Input Cell body: Processor
- Synaptic: Link
- Axon: Output
- A neuron is connected to other neurons through about *10,000 synapses*
- A neuron receives input from other neurons. Inputs are combined.

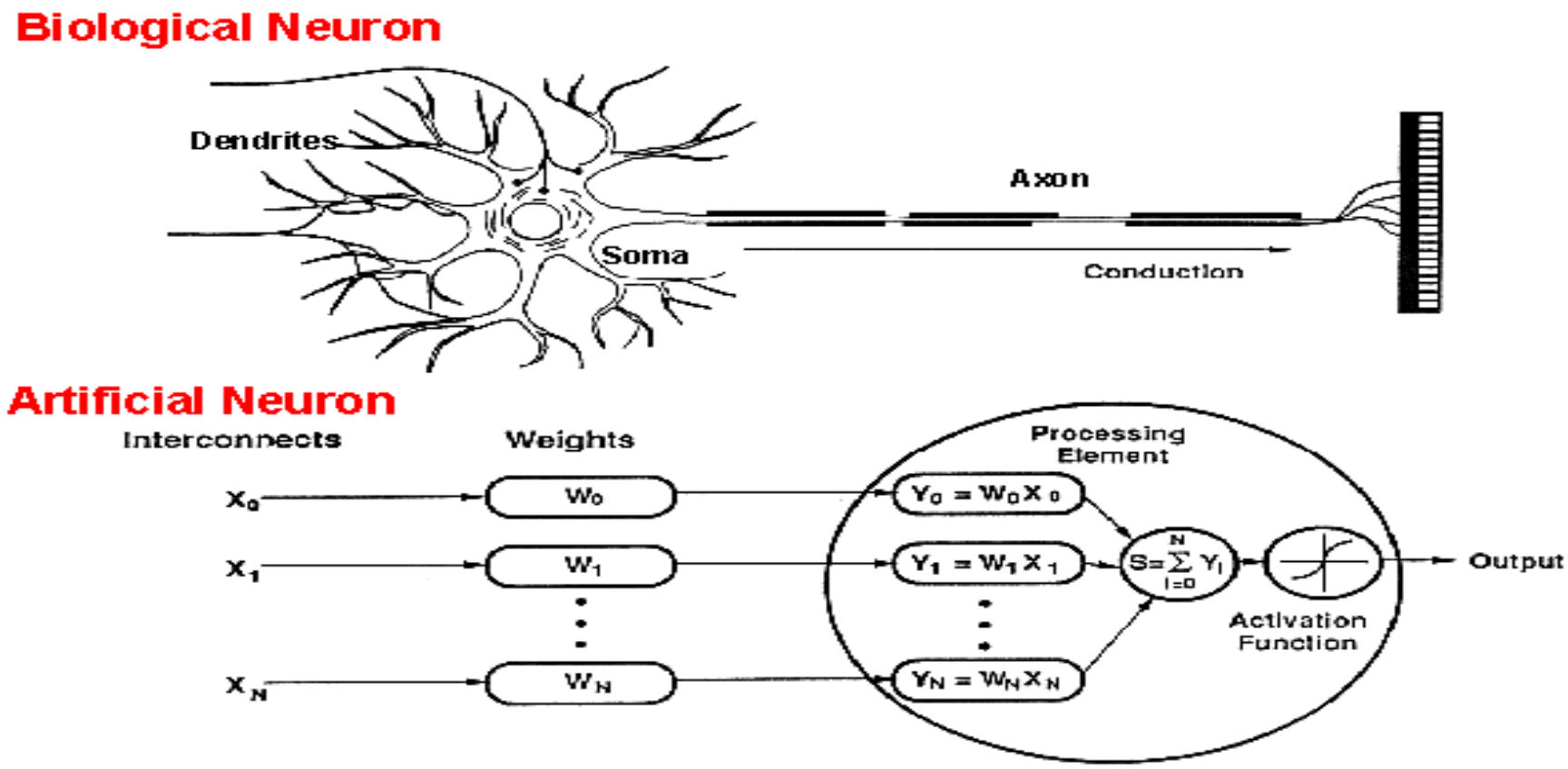


- Once input exceeds a critical level, the neuron discharges a spike - an electrical pulse that travels from the body, down the axon, to the next neuron(s)

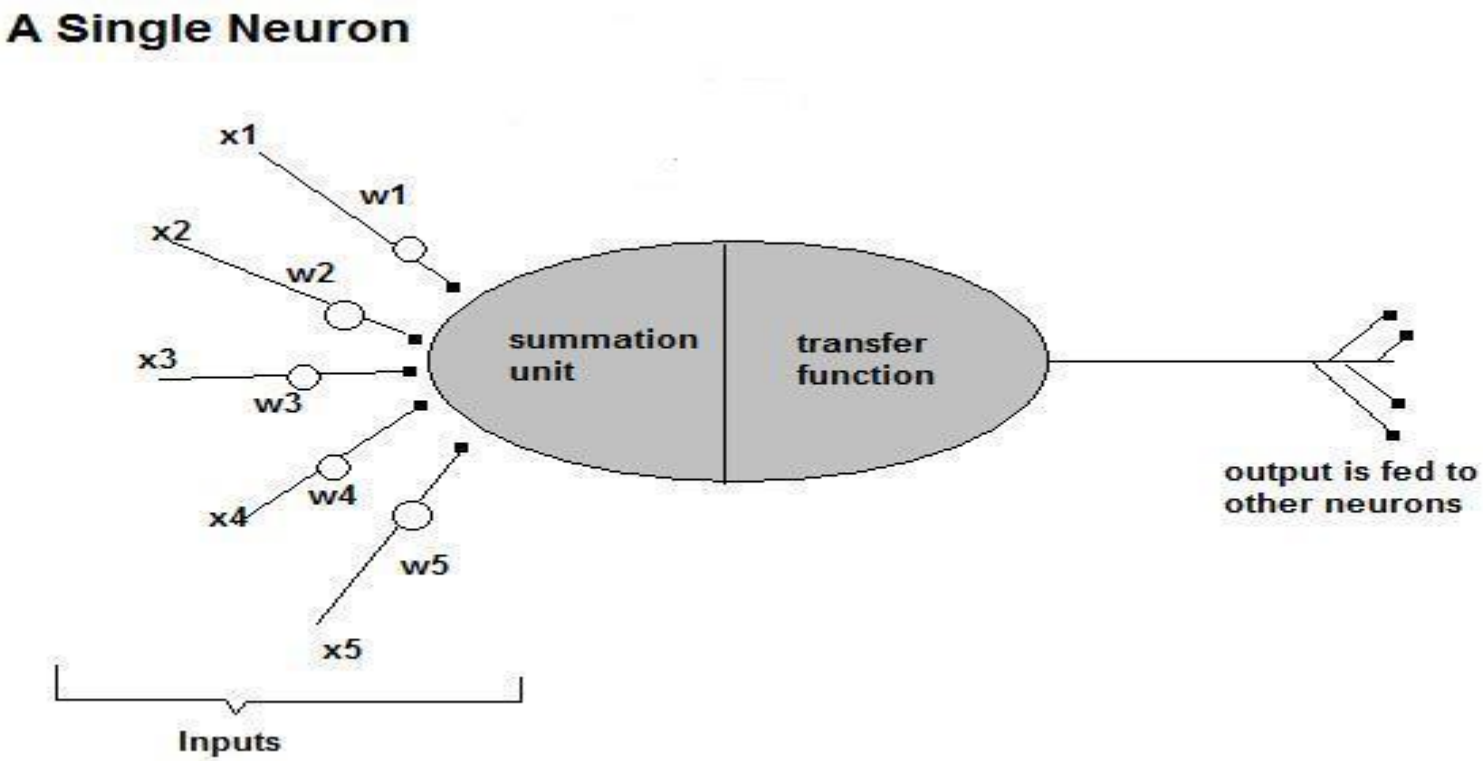
How do our brains work?

- The axon endings almost touch the dendrites or cell body of the next neuron.
- Transmission of an electrical signal from one neuron to the next is effected by neurotransmitters.
- This link is called a synapse. The strength of the signal that reaches the next neuron depends on factors such as the amount of neurotransmitter available.

Artificial Neuron



Artificial Neuron



Artificial Neuron

- Basic neural network cell consists of inputs, weights, activation function and output.
- In the case, touch decision is boolean output whereas sense organs are inputs for human nervous system.
- Weights specify strength of input on output.

Artificial Neuron

- For example, hand palm is more sensitive than the outer hand. In other words, weight of the hand palm is greater than the outer hand. Initially, weight of the detecting smoke and red color on a surface could be low value for a kid. Weight of the input is updated when kid’s hand is burnt. Thus, kid would not touch hot surface anymore.

Artificial Neuron

- Activation functions are decision making units of neural networks. They calculates net output of a neural node.
- The function produces binary output.
- That is the reason why it also called as binary step function. The function produces 1 (or true) when input passes threshold limit whereas it produces 0 (or false) when input does not pass threshold.

Artificial Neuron

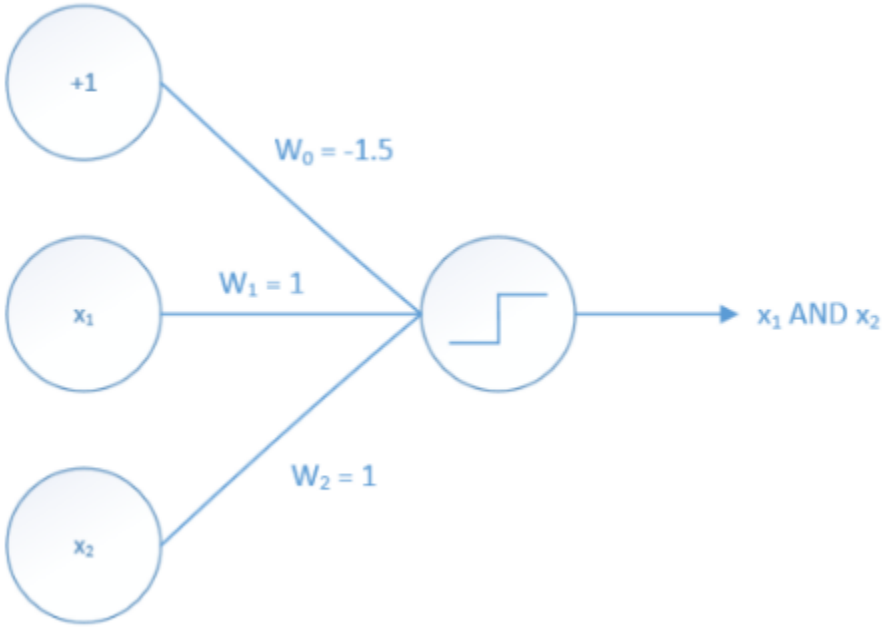
- Every logic function can be implemented by neural networks. So, step function is commonly used in primitive neural networks without hidden layer or widely known name as single layer perceptrons
- This type of network can classify linearly separable problems such as AND gate or OR gate.
- In other words, all classes (0 and 1) can be separated by a single linear line as illustrated below.

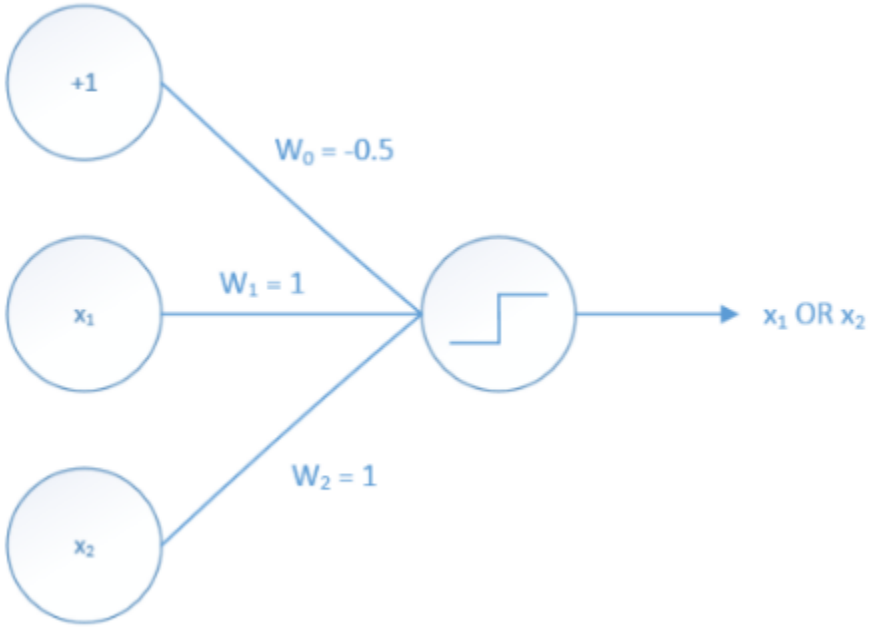
Perceptron

- A single artificial neuron that computes its weighted input and uses a threshold activation function.
- It effectively separates the input space into two categories by the hyperplane
  - $W^T X + b = 0$

Perceptron

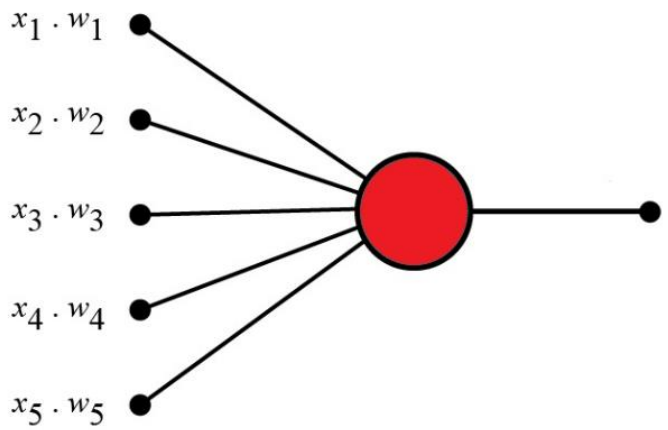
- Suppose that we picked up threshold value as 0. After then, the following single layer neural network models will satisfy these logic functions.





Perceptron

- The original Perceptron was designed to take a number of binary inputs, and produce one binary output (0 or 1).
- The idea was to use different weights to represent the importance of each input, and that the sum of the values should be greater than a threshold value before making a decision like true or false (0 or 1).



Perceptron

- Imagine a perceptron (in your brain).
- The perceptron tries to decide if you should go to a concert.
- Is the artist good? Is the weather good?
- What weights should these facts have?

Criteria	Input	Weight
Artists is Good	**x1** = 0 or 1	**w1** = 0.7
Weather is Good	**x2** = 0 or 1	**w2** = 0.6
Friend will Come	**x3** = 0 or 1	**w3** = 0.5
Food is Served	**x4** = 0 or 1	**w4** = 0.3
Alcohol is Served	**x5** = 0 or 1	**w5** = 0.4

The Perceptron Algorithm

- Frank Rosenblatt suggested this algorithm:

1. Set a threshold value (1.5)
2. Multiply all inputs with its weights
3. Sum all the results
4. Activate the output

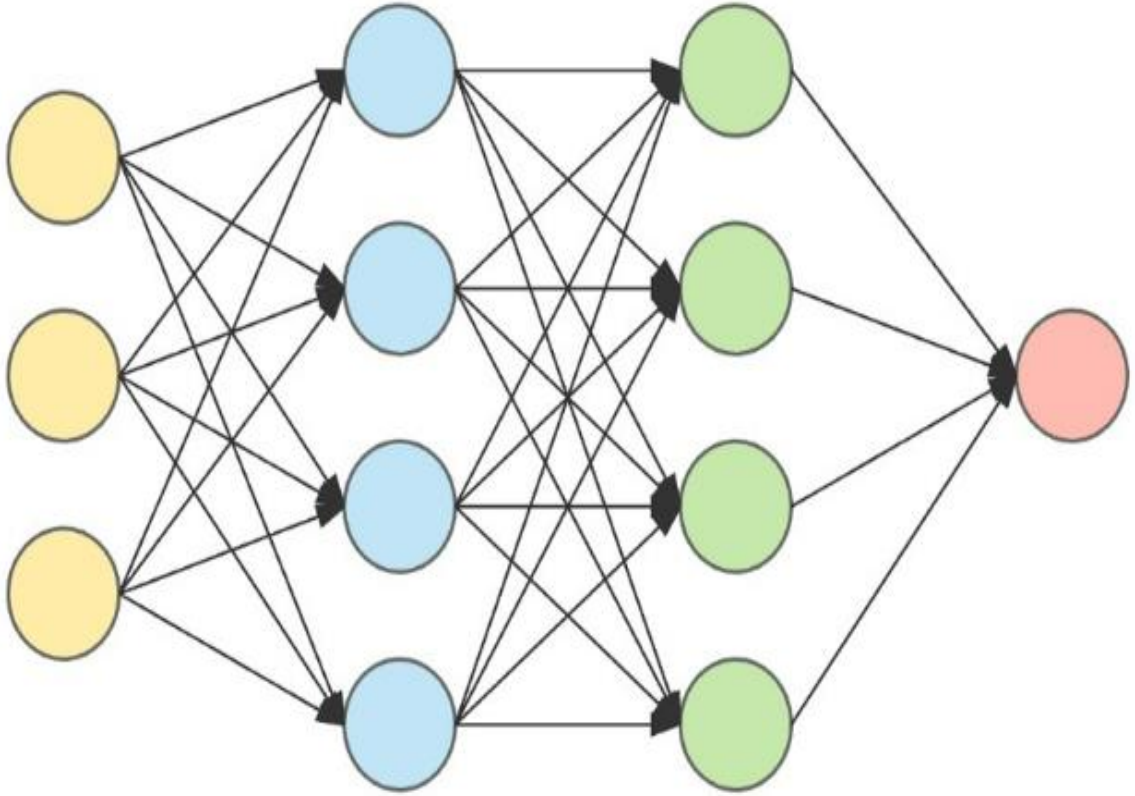
Perceptron

- x1 * w1 = 1 * 0.7 = 0.7
- x2 * w2 = 0 * 0.6 = 0
- x3 * w3 = 1 * 0.5 = 0.5
- x4 * w4 = 0 * 0.3 = 0
- x5 * w5 = 1 * 0.4 = 0.4

- 0.7 + 0 + 0.5 + 0 + 0.4 = 1.6 (The Weighted Sum)
- **Activate the Output**:
  - Return true if the sum > 1.5 ("Yes I will go to the Concert")

Perceptron

- The Perceptron defines the first step into Neural Networks.
- Multi-Layer Perceptrons can be used for very sophisticated decision making.



Perceptron

- In the Neural Network Model,
  - input data (yellow) are processed
  - a hidden layer (blue) and modified against more hidden layers (green) to produce
  - Final output (red).
- The First Layer: The 3 yellow perceptrons are making 3 simple decisions based on the input evidence. Each single decision is sent to the 4 perceptrons in the next layer.

- The Second Layer: The blue perceptrons are making decisions by weighing the results from the first layer. This layer make more complex decisions at a more abstract level than the first layer.
- The Third Layer: Even more complex decisions are made by the green perceptons.

Perceptron

- Suppose that we are going to work on AND Gate problem. The gate returns if and only if both inputs are true.
- We are going to set weights randomly. Let's say that $w_1 = 0.9$ and $w_2 = 0.9$
- We will apply 1st instance to the perceptron.
- $x_1 = 0$ and $x_2 = 0$.
- Sum unit will be 0 as calculated below
- $\Sigma = x_1 * w_1 + x_2 * w_2$
- $= 0 * 0.9 + 0 * 0.9$
- $= 0$

X_1	X_2	Y
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron

- Activation threshold would be 0.5.
- Activation unit checks sum unit is greater than a threshold.
- If this rule is satisfied, then it is fired and the unit will return 1, otherwise it will return 0.
- Sum unit was 0 for the 1st instance.
- So, activation unit would return 0 because it is less than 0.5. Similarly, its output should be 0 as well.

Perceptron

- Let’s focus on the 2nd instance.
- $x_1 = 0$ and $x_2 = 1$.
- Sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2$
- $= 0 * 0.9 + 1 * 0.9$
- $= 0.9$
- Activation unit will return 1 because sum unit is greater than 0.5.
- However, output of this instance should be 0. This instance is not predicted correctly. That’s why, we will update weights based on the error.

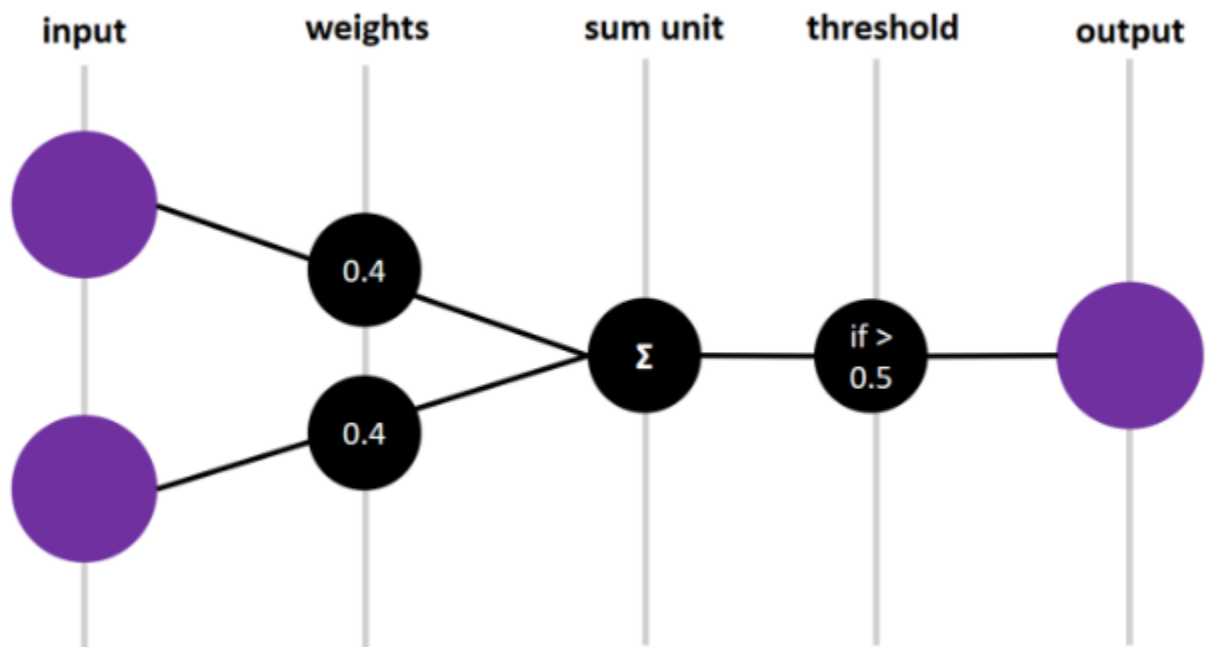
Perceptron

- ε = actual – prediction = 0 – 1 = -1
- We will add error times learning rate value to the weights. Learning rate would be 0.5.
- BTW, we mostly set learning rate value between 0 and 1.

Perceptron

- $w_1 = w_1 + \alpha * \varepsilon$
- $= 0.9 + 0.5 * (-1)$
- $= 0.9 – 0.5$
- $= 0.4$
- $w_2 = w_2 + \alpha * \varepsilon$
- $= 0.9 + 0.5 * (-1)$
- $= 0.9 – 0.5$
- $= 0.4$

- Focus on the 3rd instance. $x_1 = 1$ and $x_2 = 0$.
- Sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2$
- $= 1 * 0.4 + 0 * 0.4$
- $= 0.4$
- Activation unit will return 0 this time because output of the sum unit is 0.5 and it is less than 0.5. We will not update weights.
- Mention the 4rd instance. $x_1 = 1$ and $x_2 = 1$.
- Sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2 = 1 * 0.4 + 1 * 0.4 = 0.8$


input
weights
sum unit
threshold
output
0.4
0.4
Σ
if >
0.5

Solve the Example

To reinforce the perceptron, you should apply learning procedure for OR Gate. The gate returns 0 if and only if both inputs are 0. Do not hesitate to change the initial weights and learning rate values.

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	1